Autómatas y Control Discreto

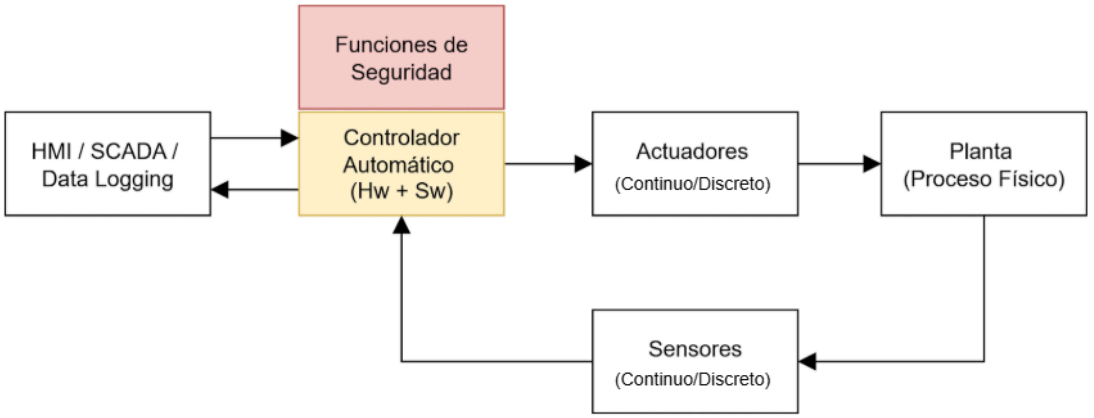
Primer Examen Parcial

Introducción a Automatismos y Control Discreto

1. [5%] **Automatismos Industriales**:

    a. Esquematizar la **estructura conceptual y componentes** genéricos de un sistema de automatización y control industrial (diagrama de bloques conceptual). Indicar que función básica cumple cada componente.

Diagrama de bloques conceptual de un sistema de automatización y control industrial:



- **HMI/SCADA/Data Logging**: Interfaz entre el operador (humano) y el sistema de control. Permite supervisar, visualizar y comandar el proceso (HMI), gestionar información en tiempo real de múltiples plantas o estaciones (SCADA) y registrar históricos de datos para análisis y trazabilidad (Data Logging).
- **Controlador Automático:** Núcleo del sistema de control. Autómata programable (PLC, PAC u otro) que ejecuta la lógica de control, tanto supervisorio como regulatorio .
    - Control supervisorio: Gestiona modos de operación, diagnóstico de fallas, monitoreo de eventos externos y coordinación de tareas secuenciales. Opera como un autómata discreto activado por eventos (FSM).
    - Control regulatorio: Se encarga de mantener variables físicas dentro de rangos deseados mediante lazos cerrados de retroalimentación (FBK) o acciones en avance (FFW). Incluye adquisición/muestreo de señales de sensores, ejecución del algoritmo de control y envío de órdenes a actuadores.
- **Funciones de Seguridad:** Módulo destinado a la protección del sistema frente a fallas (safety) o sabotajes (security). Ante una condición anómala, lleva el sistema a un estado seguro, evitando daños a personas, equipos y planta.
- **Actuadores:** Dispositivos que convierten señales eléctricas o digitales del controlador en acciones físicas sobre la planta. Pueden ser mecánicos, hidráulicos, neumáticos o eléctricos (por ejemplo: contactores, válvulas, motores). Permiten la manipulación directa de las variables de proceso.
- **Sensores:** Dispositivos de entrada que convierten variables físicas (como temperatura, presión, velocidad, etc.) en señales eléctricas o digitales, que son enviadas al controlador como retroalimentación para el lazo de control.

- **Planta:** El proceso físico real a controlar. Se trata de un sistema dinámico multivariable (MIMO), generalmente híbrido (con estados continuos y discretos) y complejo (puede ser descentralizado, distribuido y jerárquico).

    b. Describir algunas medidas de **seguridad** básicas utilizadas en automatismos industriales.

- **Vigilancia interna (Watchdog):** Monitorea el funcionamiento del controlador; si detecta que el programa deja de responder, activa una parada segura.
- **Redundancia I/O:** Duplicación de elementos críticos, que pueden ser tanto módulos de entrada como de salida.
- **Sistema separado de emergencia cableado (relé maestro de seguridad):** Circuito cableado e independiente del controlador que permite detener la planta ante una emergencia.
- **Sistema de Seguridad Intrínseca (Hw + Sw):** Integración de funciones de seguridad directamente en el controlador, con hardware y software diseñados para cumplir normas de seguridad.

2. **[UNA PREGUNTA SOBRE SISTEMAS DINÁMICOS CUYA CONSIGNA NO ESTÁ EN EL WORD]**

    a. Definición de **sistema dinámico**.

Sistema: Conjunto de elementos interrelacionados, considerado como un todo en un contexto definido, y separado de su entorno o ambiente, con el que puede intercambiar variables o señales.

Sistema dinámico: Proceso cuyas variables de estado evolucionan en el tiempo en forma demorada, es decir, con retardos (almacenamiento de energía, memoria) ante excitaciones o entradas dadas.

    b. **Clasificación** de sistemas dinámicos.

Sistemas dinámicos físicos. *Estados continuos, activados por tiempo (time-driven).*

Responden a leyes físicas y fuerzas de la naturaleza. Pueden modelarse en tiempo continuo (TC) o en tiempo discreto (TD).

- SDTC - Modelos en Tiempo Continuo: Almacenamiento e intercambio de **energía**. Las variables de estado, entrada y salida toman valores contenidos en conjuntos numéricos continuos, finitos o infinitos, y evolucionan en tiempo continuo. Se clasifican en LTI, LTV, LPV, NL, etc. **Flow variables.**
- SDTD - Modelos en Tiempo Discreto: Representación muestreada de SDTC (ZOH). Las variables de estado, entrada y salida toman valores contenidos en conjuntos numéricos continuos (o cuantizados), finitos o infinitos, y evolucionan en tiempo discreto (o muestreado). **Staircase flow variables.**

Sistemas simbólicos. *Estados discretos, activados por eventos (event-driven).*

Poseen estados (modos de operación, etapas) discretos. El cambio de un estado a otro se produce cuando ocurre un evento determinado.

- DEDS - Discrete Event Dynamic Systems: Almacenamiento (memoria) e intercambio (comunicación) de **información o datos**. Las variables de estado, entrada y salida toman valores discretos contenidos en conjuntos numéricos finitos o contables, y evolucionan en tiempo discreto, ante un evento asincrónico. **Jump variables** (“saltan” o conmutan instantáneamente de un valor a otro). Son la base de los autómatas.

Sistemas híbridos. *Activados por tiempo y por eventos.*

Combinación e interacción de un SD físico y un SD simbólico. Estados Continuos (Físicos) + Discretos (Simbólicos). Almacenamiento e intercambio de energía + información. Estos son los sistemas mecatrónicos.

3. [5%] **Autómatas finitos. Conceptos, modelado y representación**:

    a. Definir qué es un **Autómata secuencial finito** o Máquina de Estados Finitos (**FSM**) y sus componentes básicos: Estados, Acciones, Transiciones, Eventos o Condiciones. Representar gráficamente (en forma genérica).

Una Máquina de Estados Finitos (FSM) es un modelo que representa a un sistema con un número finito de estados y transiciones entre ellos. Estas transiciones se activan en respuesta a eventos específicos y depende del cumplimiento de ciertas condiciones.

A **finite state machine** is a representation of an event-driven (reactive) system.
In an **event-driven (reactive) system**, the system makes a transition from one state (mode) to another, if the condition defining the change is true.

Una Máquina de Estados Finitos (FSM) es una representación de un sistema activado por eventos (reactivos). En este tipo de sistemas, ocurre una transición de un estado (modo) a otro si la condición que define el cambio es verdadera.

Definición formal, considerando únicamente estados y evolución de los mismos:

Una FSM es una 4-tupla, $G = (Q, E, f, q_0)$, donde:

$Q = \{q_0, q_1, \dots q_s\}$: Conjunto finito de **Estados Discretos (Modos)**.
$E = \{e_1, e_2, \dots e_E\}$: Alfabeto o conjunto de **Eventos** asincrónicos (condición o etiqueta de ***Transiciones***).
$f = Q \times E \to Q$: $f(q, e) = q'$ Función de Transición (PARCIAL: NO todas las transiciones están permitidas).
q_0: Estado o Modo **Inicial**.

Nota: una n-tupla es una estructura de datos que tiene una lista (secuencia) ordenada y finita, de n elementos.

Cuando se consideran las posibles salidas (acciones) del autómata, una FSM pasa a ser una 6-tupla, $G = (Q, E, f, q_0, O, g)$, donde se agregan:

$O = \{o_1, o_2, \dots o_O\}$: **Alfabeto de Salida**: condición de ***Transiciones***
Conjunto Finito de **Acciones** de Salida.
$g = Q \to O$: Función de **Salida** para cada Estado.

(Moore)

$O = \{o_1, o_2, \dots o_O\}$: **Alfabeto de Salida**: condición de ***Transiciones***
Conjunto Finito de **Acciones** de Salida.
$g = Q \times E \to O$: Función de **Salida** para cada Estado y/o Evento.

(Mealy)

Nota: ver inciso (b) - paradigmas.

Componentes básicos:

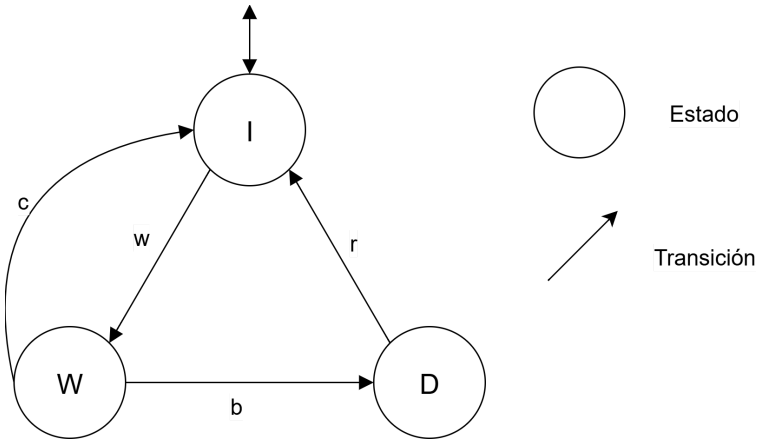
- **Estados:** Son los modos de operación del sistema.

What Is a State?

A state describes an operating mode of a reactive system. In a Stateflow chart, states represent operating modes.

- **Acciones:** Son las operaciones o tareas que ejecuta el sistema al encontrarse en un estado o al producirse una transición. Suelen ser salidas.
- **Transiciones:** Son los cambios de un estado a otro, que ocurren cuando se presenta un evento o condición específica.
- **Eventos:** Son sucesos externos o internos que disparan las transiciones. Suelen ser entradas al sistema.
- **Condiciones:** Son restricciones lógicas que deben cumplirse, junto con el evento, para que la transición ocurra.

Representación gráfica, mediante un ejemplo:



- Los estados son: {Idle (I), Work (W), Defect (D)}
- Los eventos: {w, b, r, c}
- Transiciones:

$$f(\mathrm{I}, w) = \mathrm{W};$$
$$f(\mathrm{W}, b) = \mathrm{D};\ f(\mathrm{W}, c) = \mathrm{I}$$
$$f(\mathrm{D}, r) = \mathrm{I};$$

- Estado inicial (representado mediante flecha bidireccional): I
- Considerando salidas:

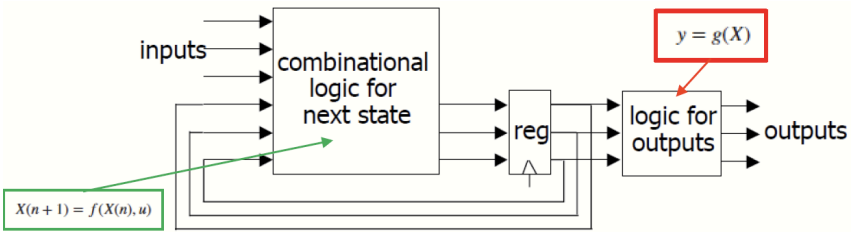
$$O = \{o_I, o_W, o_D\}$$

$$g(\mathrm{I}) = o_I;\ g(\mathrm{W}) = o_W;\ g(\mathrm{D}) = o_D$$

b. Explicar concepto y diferencias de distintos **paradigmas de autómatas: Moore vs. Mealy**.

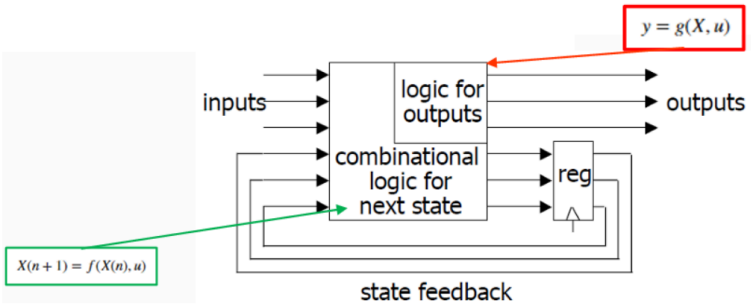
Paradigma de Moore

El valor actual de las salidas depende únicamente del estado actual del sistema. Las entradas (eventos) influencian los cambios de estado; en este sentido, pueden influenciar *indirectamente* a las salidas. Las salidas cambian en forma demorada ante un cambio en las entradas (**sincrónicas** con cambio de estados).



Paradigma de Mealy

El valor actual de las salidas depende del estado actual del sistema y/o de eventos actuales. Las entradas (eventos) influencian los cambios de estado y también pueden influenciar *directamente* e *inmediatamente* las salidas (**asincrónicas** con cambio de estados).



4. [10%] **Autómatas finitos. Representación mediante GRAFCET**:

   a. Describir sus componentes básicos de **Estructura**: Etapas o Pasos (E), Transiciones (T), Enlaces dirigidos; e **Interpretación**: Acciones (A), Receptividad (R). Indicar sus "estados" posibles.

GRAFCET (**Gra**fo **F**uncional de **C**ontrol **E**tapa-**T**ransición): Lenguaje gráfico de especificación para descripción funcional del comportamiento lógico determinista de la parte secuencial de sistemas de automatización y control.

Sus componentes básicos son:

Estructura

Describe las evoluciones posibles de la *situación* del sistema. La situación de un autómata es un mapa de todos los estados activos en un instante específico (estado global del sistema). Si el autómata no tiene concurrencia, puede haber solamente un estado activo a la vez.

- **Etapas o Pasos (E):** Representan a los estados discretos, y definen la situación actual de la parte secuencial del sistema. Inactivo/Activo.

- **Transiciones (T):** Indican la posibilidad de evolución entre dos o más etapas. Invalidada/Validada (habilitada).
- **Enlaces dirigidos:** Indican el camino de evolución posible del sistema; son las reglas de sintaxis. Siempre van desde una E a una T, o bien, de una T a una E (nunca E → E o T → T).

Interpretación

Relaciona las variables de entrada con la estructura, y la estructura con las variables de salida (acciones asociadas a cada etapa).

- **Condiciones de Transición (C):** Son evaluaciones lógicas (de expresión booleana) formadas con variables de entrada (externas) y/o variables internas del sistema, que se encuentran asociadas a cada transición. Verdadero/Falso.
- **Acciones (A):** Asignación de estado (acción continua) o asignación de evento (acción memorizada), cambio en alguna variable de salida (externa) o variable interna del sistema. Asociada a una etapa.

b. Indicar las **reglas básicas de evolución** o funcionamiento del sistema representado mediante GRAFCET.

Regla 1: Situación inicial

La situación inicial es el conjunto de etapas activas en el instante inicial del sistema (por ejemplo, al energizar el controlador secuencial). Se debe elegir como situación inicial, una situación segura.

Regla 2: Ejecución, despeje o franqueamiento de una transición

Una transición está validada (habilitada) cuando todas las etapas inmediatas precedentes se encuentran activas.

Si una transición está validada (habilitada) y su condición de transición es verdadera, entonces la transición se ejecuta (despeja, franquea) inmediatamente (en tiempo nulo).

Regla 3: Evolución de etapas activas

Ante la ejecución (despeje) de una transición, ocurre, simultáneamente, la activación de todas las etapas inmediatamente siguientes, y la desactivación de todas las etapas inmediatamente precedentes.

Regla 4: Evoluciones simultáneas

Cuando se tiene concurrencia (paralelismo), las transiciones deben ser ejecutadas simultáneamente, para tener un cambio instantáneo completo de la situación original a la nueva situación (sin intermedios).

Regla 5: Activación y desactivación simultánea de una etapa

Si una etapa activa es simultáneamente desactivada y activada, permanece activa.

Autómatas Programables - Normal IEC 61131 (Arquitecturas y Lenguajes)

5. [10%] **Funciones y Arquitecturas básicas de PLCs**:

    a. Definir qué es un **Controlador Programable (PLC)**, cuáles son sus características principales y su ámbito natural de aplicación.

Según la Electropedia (IEC), un controlador programable es un controlador basado en un microprocesador que tiene una memoria programable para el almacenamiento interno de instrucciones definidas por el usuario. Está diseñado para la automatización y control de procesos industriales.

> **programmable controller**
> microprocessor-based controller with a programmable memory for the internal storage of user-defined instructions

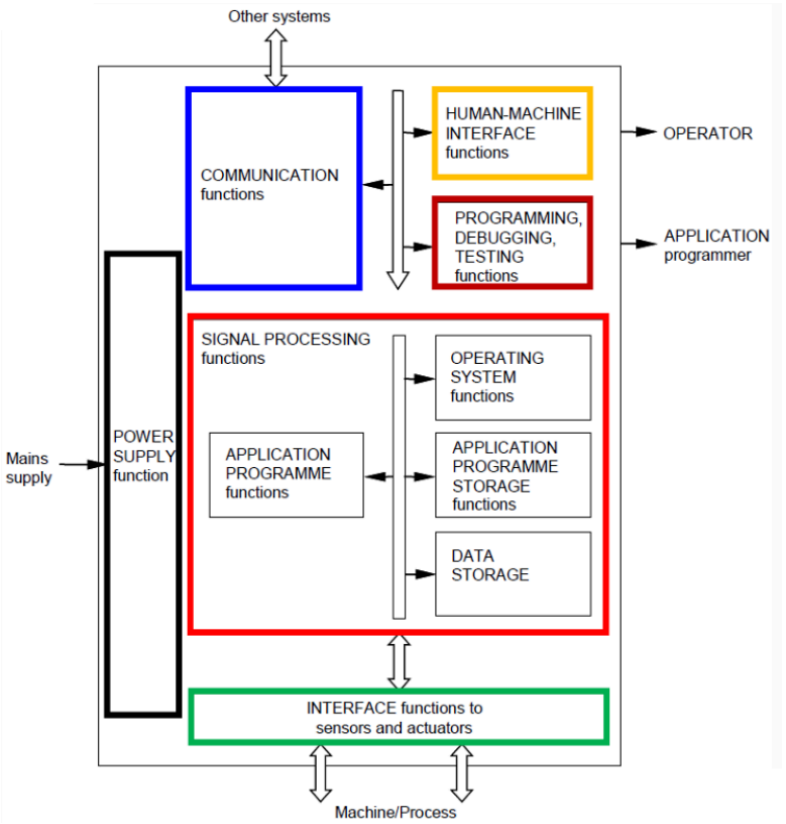
Características principales

- Las instrucciones que almacena pueden incluir funciones lógicas (combinacionales y secuenciales), funciones de temporización, conteo, cálculos, entre otros.
- Cuenta con entradas y salidas (I/O), que pueden ser digitales y/o analógicas (D/A), para comunicarse con sensores y actuadores que forman parte del sistema a controlar.
- Puede encontrarse en forma compacta o modular.

Ámbito natural de aplicación: Los PLC están diseñados para sobrevivir en ambientes industriales hostiles. En general, disponen de protecciones que evitan la entrada de polvo y líquidos a los circuitos electrónicos internos, pueden funcionar en un amplio rango de temperaturas, y presentan alta inmunidad frente a interferencias electromagnéticas, impactos mecánicos y vibraciones.

    b. Describir brevemente la **estructura funcional** y componentes funcionales básicos de un sistema de PLC y su modelo de hardware, indicados en la **Figura 1** (IEC 61131-1).

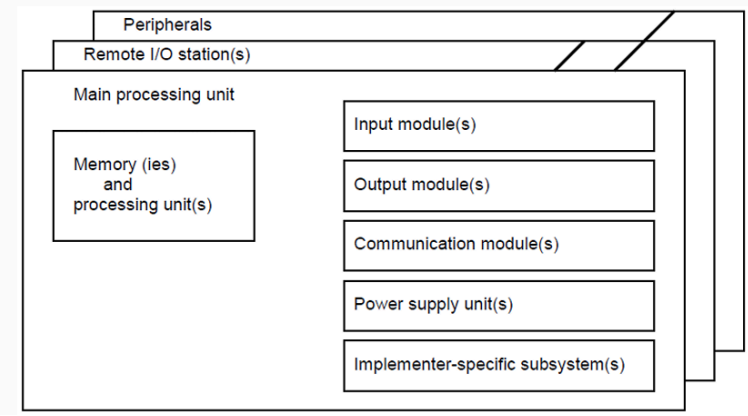
Estructura funcional básica de un Sistema de PLC:



Los componentes funcionales de un sistema de PLC se listan a continuación. Cabe mencionar que estos componentes se comunican entre sí y también con señales de la máquina o proceso que se controla.

- **CPU:** Se encarga de procesar las señales de entrada que provienen de los sensores y los datos almacenados en la memoria interna. Ejecuta el programa de aplicación de forma cíclica. Genera las señales de salida que van hacia los actuadores y actualiza los datos internos del sistema.
- **Interface I/O:** Aquí se realiza la conversión y acondicionamiento de las señales que vienen desde los sensores y van hacia los actuadores.
- **Comunicación:** Facilita el intercambio de datos con otros sistemas. Pueden incluir otros PLCs, sistemas SCADA, módulos remotos, etc. Soporta protocolos estándares de comunicación.
- **HMI:** Permite la interacción con operadores humanos.
- **Programación y depuración:** Permite la generación, carga, monitoreo y ensayo del programa.
- **Fuente de alimentación:** Proporciona la energía eléctrica necesaria para el funcionamiento del PLC. Regulación, protección contra sobrecargas.

Modelo de Hardware:

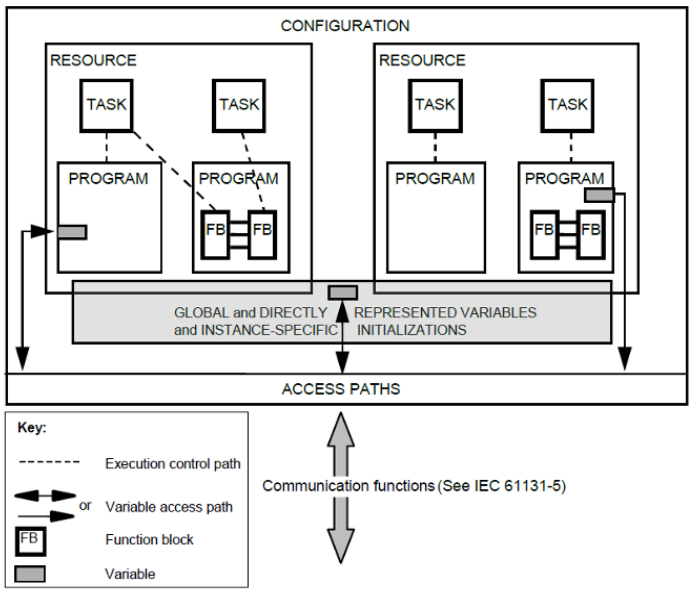


- **Peripherals** (Periféricos): Equipos externos que complementan al PLC (dispositivos de programación, interfaces hombre-máquina (HMI), impresoras, etc.). No forman parte del núcleo del PLC, pero se conectan para operación o mantenimiento.
- **Remote I/O station(s)** (Estaciones de E/S remotas): Módulos de entradas y salidas situados físicamente lejos del PLC. Se comunican mediante redes industriales, reduciendo el cableado y facilitando la expansión del sistema.
- **Main processing unit** (Unidad de procesamiento principal): Es el núcleo del PLC, donde se ejecuta el programa de usuario. Los componentes funcionales están descritos arriba.

6. [10%] **Entorno normalizado y Lenguajes de Programación (ST/SFC)**:

    a. Describir brevemente el **modelo de software** de un sistema de PLC y sus componentes básicos, indicado en la **Figura 2** (IEC 61131-3).

Modelo de software:



NOTE 1 Figure 1 is illustrative only. The graphical representation is not normative.

NOTE 2 In a configuration with a single resource, the resource need not be explicitly represented.

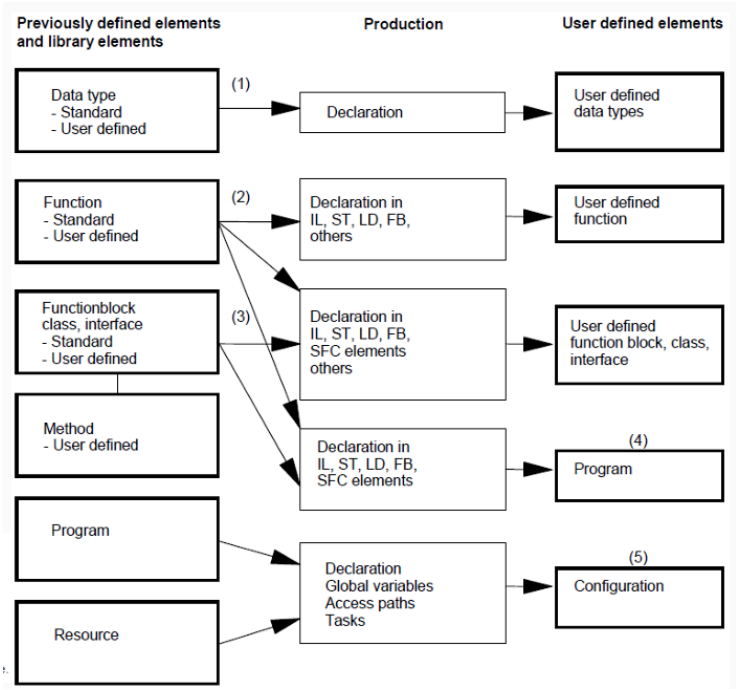
El modelo de software de un sistema PLC estandariza la manera de estructurar y programar un sistema de automatización, independientemente del fabricante. Se representa mediante niveles jerárquicos y bloques de software, compuestos por:

- **Resources** (Recursos): Así se le llama a las diferentes unidades de procesamiento o procesadores. Si el sistema cuenta con un único procesador, no es necesario representarlo de forma explícita.
- **Tasks** (Tareas): Son las unidades de ejecución. Determinan cuándo y con qué prioridad se ejecutan los programas. Pueden ser asincrónicas, periódicas o por eventos.
- **Program Organization Units** (Unidades de Organización de Programa, **POUs**): Representan al código en sí (unidad de código que puede compilarse y reutilizarse). Se clasifican en **programas** (pueden ser ejecutados en sí mismos), **funciones** (FCs, toman entradas y generan salidas, sin estado interno) y **bloques de funciones** (FBs, son como las funciones, pero además almacenan un estado interno). Las funciones y los bloques de funciones son llamados desde otros POUs para organizar y encapsular la lógica.

Además, la norma define otros elementos como:

- **Variables**: Datos utilizados por los POUs. Pueden ser globales, de programa o de bloque.
- **Access Paths** (Comunicación): Rutas de acceso a variables y funciones de comunicación para intercambiar datos entre recursos, configuraciones o dispositivos externos.
- **Configuration** (Configuración): Es el máximo nivel de un proyecto PLC. Representa todo el sistema de control que se ejecuta en un PLC o en varios PLCs interconectados.

b. Describir brevemente los **elementos de lenguajes de programación** del modelo de software de un sistema de PLC indicados en la **Figura 3** (IEC 61131-3).



La IEC 61131-3 no solo define el modelo de software, sino también los elementos de lenguaje de programación que se utilizan para crear los distintos bloques. La norma permite que estos elementos se programen en cualquiera de los lenguajes IEC:

Lenguajes Textuales:

- **IL** (Instruction List, obsoleto en ediciones recientes): estilo Assembler, basado en operaciones simples.
- **ST** (Structured Text): lenguaje tipo Pascal; usa instrucciones y estructuras de control (IF, CASE, FOR) para cálculos y lógica compleja.

Lenguajes Gráficos:

- **LD** (Ladder Diagram): usa contactos y bobinas para representar lógica de control.
- **FBD** (Function Block Diagram): permite interconectar bloques de funciones (AND, OR, temporizadores, etc.) mediante líneas de señal.
- **SFC** (Sequential Function Chart) → GRAFCET: organiza el programa en pasos y transiciones para describir secuencias de operaciones.

Adicionales:

- Extensiones orientadas a objetos: paradigma de POO (clases, herencia, encapsulamiento, polimorfismo etc.).

Los tipos de datos, funciones, bloques de función, programas, recursos y la configuración son los elementos fundamentales que el programador declara y combina en cualquiera de los lenguajes IEC para construir un proyecto de PLC estructurado y estandarizado.

Los tipos de datos, funciones y bloques de funciones pueden provenir de una librería estándar, del fabricante, o ser definidas por el usuario (derived).

Diseño de un Sistema de Control Secuencial - Problema de Aplicación:

6) [10%] **Aplicación mediante GRAFCET / SFC**:

Diseñar un programa de aplicación para el siguiente sistema de control secuencial, a implementar en un Autómata Programable (PLC), para **coordinar movimientos** en el siguiente **entorno de embalaje**, consistente en:

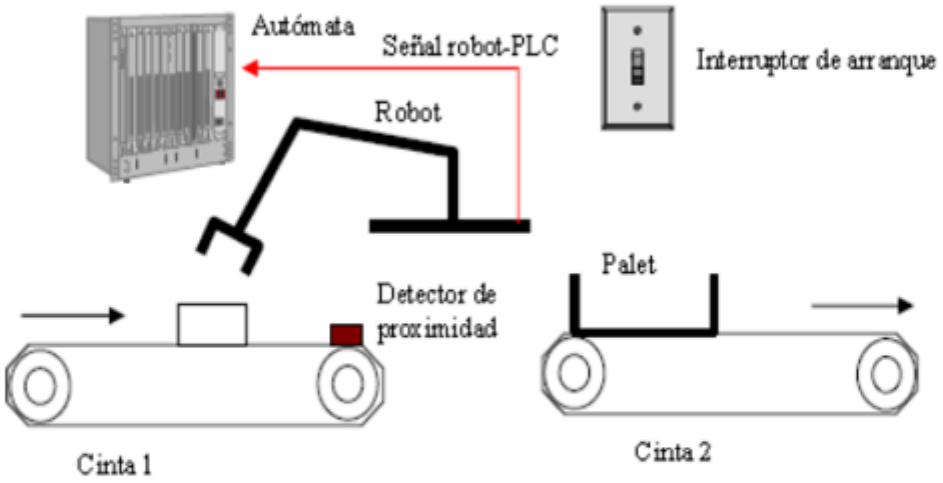
- dos cintas transportadoras, de las cuales la primera recibe cajas sueltas y la segunda entrega pallets con 10 cajas;
- un robot de transferencia de cajas entre la Cinta 1 y el pallet en la Cinta 2.

Representar dicho programa mediante dos (2) formas:

a) Diagrama de transición de estados **GRAFCET → SFC** y

b) Pseudocódigo similar a **ST, texto estructurado,**

(incluyendo declaración de **variables**/tipos de datos, **acciones** y **receptividades** correspondientes).

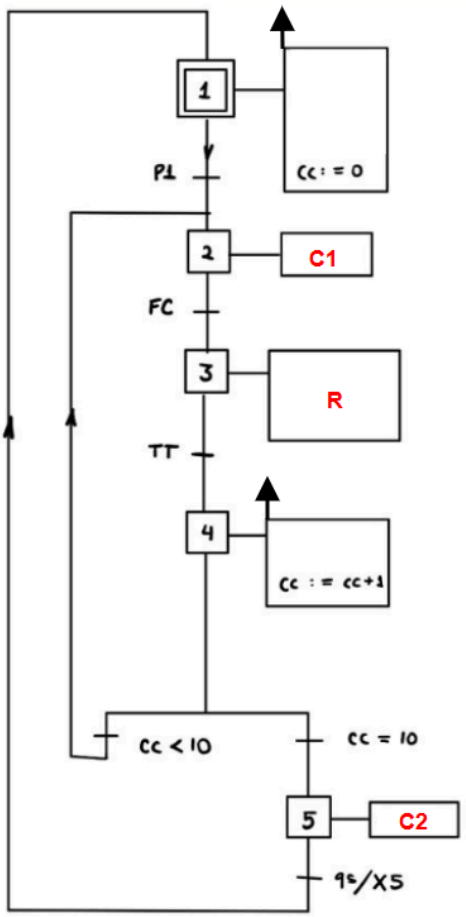


Los requerimientos operativos del sistema de control se indican a continuación:

1. Se tiene un **pulsador de arranque**, que inicia el proceso. Una vez iniciado el proceso, la **secuencia de evolución** debe ser la siguiente:
2. La Cinta 1 recibe cajas en forma asincrónica, mientras el robot y el pallet están detenidos en su posición original.
3. Cuando una caja llega al extremo de la cinta 1, es detectada por el sensor de proximidad. En ese momento el autómata debe detener la cinta 1.
4. Con la cinta 1 detenida, el autómata activa al robot, que debe tomar la caja y transferirla al pallet en la Cinta 2. Una vez terminada la operación del robot y regresado a su posición original de espera, el robot indica este estado al autómata (transferencia terminada).
5. El autómata activa nuevamente la cinta 1, a la espera de una nueva caja, mientras se hayan colocado menos de 10 cajas.
6. Cuando el robot ha colocado 10 cajas en el pallet, el autómata activa la cinta 2 durante 9 s.
7. Una vez concluido esto, el sistema queda en espera de un nuevo pallet a ser colocado manualmente por un operario, quien debe dar una nueva orden de inicio con el pulsador de arranque. Se vuelve así al paso 1.

GRAFCET

VARIABLE	DESCRIPCIÓN	E/S	TIPO	
C1	Cinta 1	S	Bool	
C2	Cinta 2	S	Bool	
P1	Pulsador de arranque	E	Bool	1 → Start
FC	Sensor de proximidad	E	Bool	1 → Objeto cerca
R	Robot	S	Bool	
TT	Indicación de transferencia del robot	E	Bool	
CC	Cantidad de cajas	Variable del sistema	Int	



Nota: Gracias pibes norma. **LOS PREDICADOS "CC < 10" Y "CC = 10" VAN ENTRE CORCHETES**.

TEXTO ESTRUCTURADO

```
PROGRAM PLC_PRG
VAR
    // ENTRADAS
    P1 : BOOL := FALSE;      // Pulsador de arranque
    FC : BOOL := FALSE;      // Sensor de proximidad
    TT : BOOL := FALSE;      // Indicador de fin de tarea del robot

    // SALIDAS
    C1 : BOOL := FALSE;      // Cinta 1
    C2 : BOOL := FALSE;      // Cinta 2
    Rb : BOOL := FALSE;      // Robot

    // VARIABLES DEL SISTEMA
    STEP : INT := 0;         // Indicador de pasos/estados
    CC : INT := 0;           // Contador de cajas
    TIM : TON;               // Temporizador para C2
END_VAR
```

```
CASE STEP OF
    0:
        // Los estados iniciales de las variables ya están declarados
        CC := 0;          // Siempre comienza a contar desde 0
        // Se espera a detectar el pulsador de arranque, P1
        IF P1 THEN
            STEP := 1;    // Siguiente paso
        END_IF
    1:
        C1 := TRUE;       // Se activa la Cinta 1
        // Se espera a la senal del sensor de proximidad, FC
        IF FC THEN
            C1 := FALSE;  // Se frena la Cinta 1
            STEP := 2;    // Siguiente paso
        END_IF
    2:
        Rb := TRUE;       // Empieza la rutina del Robot
        // Se espera a la senal del robot, TT
        IF TT THEN
            Rb := FALSE;  // Frena el Robot
            CC := CC + 1; // Aumenta el contador de cajas

            IF (CC < 10) THEN
                STEP := 1; // Vuelve al paso 1
            ELSE
                STEP := 3; // Cuando se acumulan 10 cajas, avanza al paso final
            END_IF
        END_IF
    3:
        C2 := TRUE;        // Se activa la Cinta 2
        TIM(IN := TRUE, PT := T#9S);   // Temporizador de 9 segundos
        IF TIM.Q THEN
            TIM(IN := FALSE);
            C2 := FALSE;   // Se frena la Cinta 2 al cumplirse el tiempo
            STEP := 0;     // Vuelve al paso 0, debe esperar un nuevo arranque para continuar
        END_IF;
END_CASE
```

Nota: Por las dudas, leer lo de Functional Safety del final de la U2 y mirar por encima la U3.